

# Sri Lanka Business Intelligence Copilot

Final project report | AI Engineering Bootcamp | 5 September 2026

## 1 Project purpose and scope

I developed a web application that combines research across official Sri Lankan business documents with analysis of uploaded land plans. The system provides source information alongside its answers, supports conversational follow-ups and produces business recommendations from retrieved evidence. A separate vision workflow extracts land information and adds geographic context. The completed retrieval benchmark covers 30 tasks and compares the implemented pipeline with a simpler vector-only baseline [1, 2].

The intended users are entrepreneurs, investors and researchers conducting preliminary business or land feasibility studies. Their evidence is distributed across institutional reports, while survey plans add visual information that text-only search cannot fully interpret. The project therefore prioritizes inspectable sources and clear distinctions between observations and model interpretation. Ownership, legal boundaries, market valuation and planning approval remain outside its verification capabilities.

## 2 Architecture and knowledge preparation

A Next.js and React frontend communicates with a FastAPI backend. PostgreSQL stores documents, chunks and conversation messages, with pgvector providing vector search. The backend exposes agent-routed questions, direct RAG, business advice, land analysis and conversation-history endpoints. LangSmith instruments the main workflows. The application runs locally through the browser, which is an accepted deployment mode in the project brief [3].

The ingestion workflow collects PDFs, extracts text, splits it into chunks, creates embeddings with text-embedding-3-small and stores the results with document metadata. Configured sources include the Central Bank of Sri Lanka, Board of Investment, Department of Census and Statistics, Export Development Board and Department of Agriculture. Metadata retains title, publisher, category, publication information and original URL for citations and evaluation.

| Layer                   | Implementation and responsibility                |
|-------------------------|--------------------------------------------------|
| Interface and API       | Next.js 16, React 19, TypeScript and FastAPI     |
| Storage and retrieval   | PostgreSQL, pgvector and OpenAI embeddings       |
| Decision and generation | Structured LLM routing, RAG and Business Advisor |
| Land and observability  | Vision, Nominatim, Overpass and LangSmith        |

### 3 Agent routing and document answering

Ask AI sends the question and conversation ID to /agent. The Business Copilot Agent loads stored messages and contextualizes follow-ups when history exists. A structured LLM decision selects knowledge\_search for factual questions or business\_advisor for recommendations, comparisons and feasibility requests. The decision includes a short routing explanation. Supplying a land report instructs the router to choose the advisor. The agent saves the original question and final response to conversation memory [1].

Knowledge Search uses the RAG pipeline. Vector similarity retrieves 20 candidate chunks, a cross-encoder/ms-marco-MiniLM-L-12-v2 reranker scores them and the best three chunks form the final context. The model receives this evidence with the question and grounding instructions. The response groups the selected chunks by document and returns source metadata and relevance scores. Source relevance scores describe retrieval signals and are not probabilities that the answer is correct.

Business Advisor retrieves official-document evidence and optionally combines it with a structured land report before generating a recommendation. The dedicated advisor interface calls /business-advice directly. Consequently, the land-to-advisor button can execute advice without a Choose Agent Tool span; that routing span belongs specifically to /agent. The system implements a bounded choice between two tools, rather than an unrestricted autonomous planning loop.

### 4 Multimodal land workflow

Land Intelligence accepts PDF, PNG and JPEG documents. The parser prepares page images and a GPT-4o vision call extracts boundaries, measurements, roads, survey identifiers and location clues. The workflow validates the resulting JSON. When location information is available, an LLM normalizes it into a search query and Nominatim resolves it to coordinates. Overpass retrieves nearby OpenStreetMap features within approximately three kilometres, including roads and facilities.

The workflow distinguishes broad administrative matches from more specific locations and reports match quality. Nearby distances are straight-line estimates from the geocoded point, not verified travel distances from the parcel. Business analysis can continue from document evidence when geographic information is unavailable. The report builder keeps extracted observations, external map evidence and inferred opportunities or risks separate.

### 5 Tracing and user experience

LangSmith records the Business Copilot Agent root, question contextualization, tool choice, retrieval, reranking and model generation. The land trace contains parsing, vision extraction, location normalization, geolocation, nearby intelligence, business analysis and report construction. Inputs, outputs and timing support inspection of actual behaviour. Conversation history, a fixed prompting area and answer-level source panels support extended research sessions. The submitted slides include a real agent trace and interface evidence.

## 6 Evaluation methodology and results

The evaluation report generated on 27 August 2026 contains 30 retrieval tasks across eight categories. Each task specifies expected source authorities. Metadata matching checks aliases against source fields such as title, filename, publisher, category, publication information and URL. The comparison uses the same vector candidates for both systems. Table 1 reproduces the stored results [2].

| System              | Tasks | Hit@3 | MRR   | Precision@3 |
|---------------------|-------|-------|-------|-------------|
| Vector only         | 30    | 70.0% | 0.611 | 0.556       |
| Vector and reranker | 30    | 66.7% | 0.600 | 0.400       |

Table 1 Recorded retrieval benchmark comparison

Hit@3 checks whether an expected authority appears among the first three unique document results. MRR rewards a higher rank for the first relevant authority. Precision@3 divides the relevant document count by three, including when fewer than three documents are returned. These metrics assess authority retrieval; they do not establish factual correctness or completeness of the generated answer.

Vector-only retrieval achieved 21 successful tasks, compared with 20 for the reranked system. The 3.3 percentage-point Hit@3 difference is one question, so the small sample does not establish a general ranking advantage. Investment achieved 5/5 hits; agriculture and exports each achieved 4/5; economy 3/5; statistics 1/5; tourism 2/3; cross-domain 1/1; and tax/compliance 0/1. Small category samples require cautious interpretation.

## 7 Failure analysis and benchmark limitations

Failures include expected DCS, IMF, Customs or IRD authorities being replaced by related sources such as CBSL, BOI or EDB. Some expected authorities are outside the five configured ingestion sources. These cases indicate a corpus-coverage mismatch as well as possible ranking errors. Publisher matching is only a proxy for passage relevance: a document from another authority may still contain useful evidence, and a document from the expected authority may not answer the question.

The comparison also reflects different document selection policies. The baseline searches across 20 candidate chunks until it finds three unique documents; the current pipeline selects three reranked chunks and then deduplicates them. Several chunks from one document can therefore reduce document diversity and Precision@3. A future controlled experiment should equalize the chunk and unique-document budgets before attributing the difference solely to the cross-encoder.

Six Land Intelligence scenarios are defined separately, but no executed multimodal benchmark results are included in the current report. Optional answer grading is implemented through --grade-answers but has not been run in the stored evaluation. Human review of groundedness, correctness, source relevance and land uncertainty remains a priority. The current results support a retrieval comparison, not a claim of measured end-to-end answer accuracy.

## 8 Operation and reproducibility

The repository README describes configuration, architecture, endpoints, evaluation and a demonstration sequence. Local operation requires a supported Python environment, Node.js, PostgreSQL with pgvector and an OpenAI key. LangSmith credentials are optional for tracing. The backend runs with Uvicorn and the frontend with `npm run dev`. Source crawling or an existing manifest and document collection must precede ingestion. Database contents and secret environment files are not part of the source submission [1].

The final audit passed frontend ESLint and a Next.js production build using webpack, including TypeScript checking and static page generation. Five deterministic metric tests passed through direct invocation. A fresh backend pytest run could not start because pytest is absent from the local virtual environment, although it is listed in `backend/requirements.txt`. The full suite remains unverified. The presentation has been delivered, and the submission checklist records the remaining evidence and reproduction checks.

## 9 Limitations and next steps

Retrieval quality depends on corpus coverage, chunk selection and metadata accuracy. The immediate improvements are to add missing authorities, review source labels and compare reranking under equal document budgets. Larger category samples, human-reviewed answers and executed land scenarios would provide stronger quality evidence. Actual model expenditure and comprehensive latency benchmarks have not been recorded in the submitted evaluation.

Land extraction depends on scan clarity, and geocoding may resolve only an approximate area. OpenStreetMap data may omit facilities or contain outdated entries. Uploaded plans and traces can contain sensitive information. A wider release would require stronger access controls, upload validation, file retention rules and trace redaction. The present system is intended for preliminary research, with professional verification required before property or investment decisions.

The application demonstrates integrated retrieval, model-selected tools, persistent memory and meaningful image analysis with inspectable execution. Its measured retrieval weaknesses give concrete priorities for improvement. Local browser deployment satisfies the course deployment requirement; a publicly reachable application would be an optional extension [3].

## References and submission evidence

[1] Project source and README. Sri Lanka Business Intelligence Copilot. Submission snapshot at commit `adbebd6` on branch `fine-tune`, audited 5 September 2026. Repository: <https://github.com/mithilamp/sl-business-intelligence-copilot>

[2] Retrieval evaluation report and `evaluation_results.json`, generated 27 August 2026. Included with `benchmark_questions.json` and `land_intelligence_tasks.json` in `backend/app/evaluation` and the submission evidence folder.

[3] AI Engineering Bootcamp. Final Project Overview and Project Menu. Core requirements and deliverables, pages 2 to 4. Local deployment is acceptable; a public URL is a bonus.

[4] Final presentation, 16 slides, September 2026. Includes interface screenshots, an agent trace, architecture, the retrieval comparison and a demonstration sequence.